Polymorphism in Java

Compile-time (Overloading) vs Runtime (Overriding) & Dynamic Dispatch

Topic Overview

Polymorphism allows objects to take on many forms. In object-oriented programming, it is primarily categorized into two distinct types:

- **Compile-time Polymorphism (Static Binding):** Achieved via *Method Overloading*, where multiple methods share the same name within a class but have different parameter lists. The compiler determines which method to call at compile time.
- **Runtime Polymorphism (Dynamic Binding):** Achieved via *Method Overriding*, where a subclass provides a specific implementation of a method that is already defined in its parent class. The Java Virtual Machine (JVM) decides which method executes at runtime based on the actual object instance.

Task Solutions

Task 1: Compile-time Polymorphism – Overloaded `processPayment()`

```
class PaymentProcessor {
    /* Version 1: processPayment(amount) */
    void processPayment(double amount) {
        System.out.println("[Version 1: processPayment(amount)]");
        System.out.println("Final Amount: Rs." + amount + "\n");
    }

    /* Version 2: processPayment(amount, couponCode) - OVERLOADED */
    void processPayment(double amount, String couponCode) {
        double discount = 0;
        if (couponCode.equals("SAVE10")) {
            discount = amount * 0.10;
        } else if (couponCode.equals("FLAT50")) {
            discount = 50;
        }
        double finalAmount = amount - discount;
        System.out.println("[Version 2: processPayment(amount, couponCode)]
            ");
        System.out.println("Coupon: " + couponCode + " | Discount: Rs." +
            discount);
        System.out.println("Final Amount: Rs." + finalAmount + "\n");
    }
}

public class Task1_PaymentProcessor {
    public static void main(String[] args) {
        PaymentProcessor payer = new PaymentProcessor();
        payer.processPayment(1000);              // Calls Version 1
        payer.processPayment(1000, "SAVE10");    // Calls Version 2
        payer.processPayment(1000, "FLAT50");    // Calls Version 2
```

```
    }
}
```

Task 2: Runtime Polymorphism – uploadContent() Overriding

```
class SocialMediaUploader {
    void uploadContent() {
        System.out.println("Uploading content to social media...");
    }
}

class InstagramUploader extends SocialMediaUploader {
    @Override
    void uploadContent() {
        System.out.println("Uploading a Reel/Story to Instagram (max 90
            seconds, vertical video)...");
    }
}

class YouTubeUploader extends SocialMediaUploader {
    @Override
    void uploadContent() {
        System.out.println("Uploading a full video to YouTube (any length,
            thumbnail + title)...");
    }
}

public class Task2_Uploaders {
    public static void main(String[] args) {
        SocialMediaUploader insta = new InstagramUploader();
        SocialMediaUploader ytube = new YouTubeUploader();

        insta.uploadContent();   // Instagram version runs
        ytube.uploadContent();   // YouTube version runs
    }
}
```

Task 3: Flipkart-style Search – Overloaded searchProduct()

```
class FlipkartSearch {
    private String[][] products = {
        {"iPhone 15", "Electronics"},
        {"Samsung TV", "Electronics"},
        {"Nike Shoes", "Fashion"},
        {"Levi's Jeans", "Fashion"},
        {"Prestige Cooker", "Home & Kitchen"}
    };

    /* Version 1: search by name only */
    void searchProduct(String name) {
        System.out.println("Searching for name: \"" + name + "\"");
        for (String[] p : products) {
            if (p[0].toLowerCase().contains(name.toLowerCase())) {
```

```
                System.out.println("   Found: " + p[0] + " (Category: " + p
                    [1] + ")");
            }
        }
        System.out.println();
    }

    /* Version 2: search by name + category */
    void searchProduct(String name, String category) {
        System.out.println("Searching \"" + name + "\" in category \"" +
            category + "\"");
        for (String[] p : products) {
            if (p[0].toLowerCase().contains(name.toLowerCase()) && p[1].
                equalsIgnoreCase(category)) {
                System.out.println("   Found: " + p[0] + " (Category: " + p
                    [1] + ")");
            }
        }
        System.out.println();
    }
}
```

Task 4: Dynamic Method Dispatch

```
class MusicPlayer {
    void play(String song) {
        System.out.println("Playing: " + song);
    }
}

class SpotifyPlayer extends MusicPlayer {
    @Override
    void play(String song) {
        System.out.println("Streaming on Spotify: " + song);
    }
}

public class Task4_MusicPlayer {
    public static void main(String[] args) {
        MusicPlayer player = new SpotifyPlayer();
        player.play("Believer");  // Outputs: Streaming on Spotify:
            Believer
    }
}
```

Summary of Polymorphism Types

#	Concept	Type	Resolution Time
1	`processPayment()`	Overloading	Compile-Time
2	`uploadContent()`	Overriding	Runtime
3	`searchProduct()`	Overloading	Compile-Time
4	`MusicPlayer/Spotify`	Overriding	Runtime (Dynamic Dispatch)

Key Takeaway: Overloading lets methods share names with distinct signatures resolved by the compiler, whereas overriding enables dynamic runtime behavior via subclass specialization.